MobiCloud — Oral Defense Q&A

Simple questions the jury is likely to ask. Focus on **"Why did you choose..."**, broad understanding, and **"What if..."** scenarios.

1 — PROJECT & MOTIVATION

Q1. In one sentence, what problem does MobiCloud solve? General

MobiCloud helps users store files securely on a peer-to-peer network of mobile phones instead of using a central cloud server.

Q2. Why did you choose a P2P approach rather than a simple client-server model? Why

A client-server model is expensive to host and has a single point of failure. P2P uses the storage and network of the phones themselves, which protects privacy and removes server costs.

Q3. Why target Android specifically, and not a desktop or web application? Why

Almost everyone has a smartphone with them all the time. Also, Android has a secure hardware enclave (TEE) to protect cryptographic keys. Web apps cannot offer this level of security.

Q4. What are the main limitations of your current prototype? General

Currently, we use a central relay server to help phones connect. Also, the app only supports a small number of devices for testing, and we need a more user-friendly interface.

2 — ARCHITECTURE & DESIGN CHOICES

Q5. Why did you introduce a Super-Peer role instead of a fully flat P2P network? Why

Smart phones have limited battery and CPU. If every phone had to coordinate everything, they would lose battery quickly. The Super-Peer does the heavy work of coordinating the network, while regular nodes can save power.

Q6. Why did you choose the Bully algorithm for Super-Peer election? Why

The Bully algorithm is simple and reliable. It selects the best Super-Peer based on a stability score. This ensures that the most stable phone is always in charge.

Q7. Why did you add a relay server if the goal is decentralisation? Why

Phones on 4G or home Wi-Fi are behind firewalls (NAT) and cannot connect directly to each other. The relay helps them connect. It is only a bridge; it never reads or stores the files.

Q8. Why use Clean Architecture (domain / data / presentation layers) for an Android app? Why

Clean Architecture separates the core logic from Android-specific code. This makes the app easy to test and update. It also let us run P2P simulations on a computer without using a slow Android emulator.

Q9. You used Kotlin Coroutines and Flow throughout the app. Why not threads or callbacks? Why

P2P apps need to manage many network connections and database tasks at the same time. Normal threads use too much memory. Coroutines are very light, and Flow helps us handle live network data easily without messy code.

3 — STORAGE & REDUNDANCY

Q10. Why did you choose Erasure Coding (Reed-Solomon) instead of simple replication? Why

If we copy the whole file 3 times, it uses 3 times more storage. With Reed-Solomon (k=3, n=2), we divide the file into 3 parts and add 2 backup parts. The total storage is only 1.67 times the file size. We get the same safety but use half the storage.

Q11. What happens if a peer that holds a fragment goes offline permanently? General

The Super-Peer checks the network every 10 seconds. If a node goes offline, the Super-Peer finds which fragments are missing. It then tells another node holding a copy of that fragment to send it to a new node, keeping the file safe.

Q12. Why did you use a local Room (SQLite) database instead of keeping everything in memory? Why

Android can close apps at any time to save memory. If we kept the network data in RAM, we would lose everything on restart. Storing it in Room (SQLite) database keeps the data safe so the app can restart and connect quickly.

4 — SECURITY & PRIVACY

Q13. Why did you choose AES-256-GCM for fragment encryption? Why

AES-256-GCM is a secure encryption algorithm. It encrypts the files and checks if they were modified or corrupted. If someone changes the encrypted file, decryption fails immediately. It is also very fast because modern phone processors support it in hardware.

Q14. What is the Hashcash mechanism and why did you add it? General

Hashcash is a simple proof-of-work. To join the network, a phone must solve a small math puzzle (finding a SHA-256 hash with 18 zero bits). This takes about 1 second. It prevents

attackers from spamming the network with thousands of fake accounts because it would require too much computer power.

Q15. How does MobiCloud ensure that the relay server cannot read the stored files? General

We encrypt the file fragments on the phone before sending them over the network. The relay server only sees encrypted data and does not have the keys to decrypt it. Even if the relay server is hacked, the attacker cannot read anything.

Q16. If your system is Peer-to-Peer, why do you need Supabase and a WebSocket Relay? Isn't this centralized? Why

No. They only handle connections and backup metadata, not your files. The WebSocket Relay helps phones behind firewalls connect to each other. Supabase only stores encrypted identity files. If both servers go down, phones on the same Wi-Fi can still find each other and share files directly.

Q17. How do you guarantee that Supabase employees or database hackers cannot read the backed-up private keys? General

The private keys are encrypted on the phone before they are uploaded to Supabase (Zero-Knowledge). The app encrypts them using AES-256-GCM with a key derived from the user's recovery password. Supabase never sees the password, so no one can decrypt the backup in the cloud.

5 — GOSSIP & SYNCHRONISATION

Q18. What is the Gossip protocol and why did you use it to synchronise the DHT? General

The Gossip protocol works like sharing a rumor. Every few seconds, a phone shares its database with two random neighbors. They do the same, and soon the whole network is

updated. There is no master server. It is decentralized, self-healing, and works very well as the network grows.

Q19. Why did you use a Bloom Filter inside the Gossip protocol instead of sending the full DHT? Why

Sending the entire list of files over the network uses too much bandwidth. Instead, we use a Bloom filter, which is a tiny summary of the data (only 128 bytes). The receiver checks the summary to find what is missing and only requests the missing files.

Q20. Can a Bloom Filter give wrong answers? How does your system handle that? General

Yes, a Bloom filter can have false positives (saying it has a file when it actually does not). However, it never has false negatives. This means we might delay syncing a file for a few seconds, but we will never lose data. The next update cycle will fix it.

6 — SIMULATION & VALIDATION

Q21. Why did you use PeerSim for simulation instead of testing on real devices? Why

Testing a P2P network with 100 phones is not possible for a student project. PeerSim is a simulator that runs on a computer. It lets us test how thousands of nodes behave, how they handle disconnects, and if the system works correctly. We also tested the app on 3-4 real devices.

Q22. What did your simulation tell you about the system's behaviour under churn? General

Our tests showed that even if 30% of nodes leave the network suddenly, the system still works. If the Super-Peer crashes, a new one is elected in less than 6 seconds. The system only becomes unstable if more than 50% of devices disconnect at the same time.

Q23. How did you test the security of the system? General

We secured the network in four ways: (1) every important message is signed cryptographically; (2) messages have a 30-second timestamp to prevent reuse (replay attacks); (3) we ignore messages from unknown devices; and (4) only active group members can update the file list.

7 — OPEN QUESTIONS & FUTURE WORK

Q24. If you had 6 more months, what would you improve first? General

I would improve three things: (1) remove the central relay server and replace it with a fully decentralized connection layer; (2) add support for file versions to prevent users from overwriting each other's changes; and (3) measure the exact battery usage on real phones to optimize performance.

Q25. How does MobiCloud compare to existing systems like IPFS or Bittorrent? General

IPFS and BitTorrent are built for computers and use too much battery and memory for a phone. MobiCloud is designed from scratch for mobile devices. It is lightweight, fully encrypted, and built for private sharing between friends rather than public downloads.

Q26. Is MobiCloud ready for production use? General

No, it is a research prototype. The relay server runs on a free cloud service and the cluster size is limited for testing. It does not have a full user interface or account management. These are engineering features we would add for a real product.

---

8 — WHAT IF...

Q27. What if the relay server goes down completely? What If

Nodes on the same Wi-Fi network can still find each other using local multicast (mDNS) and transfer files directly. Devices on 4G will lose connection until the relay is back online. No files are lost because the fragments are stored on the phones.

Q28. What if the Super-Peer of a cluster crashes suddenly? What If

If a phone stops receiving updates from the Super-Peer for 2 minutes, it starts a new election. The phone with the best stability score sends an election message. If no other phone has a higher score after 3 seconds, it becomes the new Super-Peer. The entire recovery takes less than 2 minutes.

Q29. What if more than n=2 peers holding fragments go offline at the same time? What If

If 3 or more nodes holding fragments disconnect at the same time, we cannot rebuild the file. To prevent this, our repair system checks the network every 10 seconds. If a node goes offline, the system immediately copies its fragment to a new node before more devices can disconnect.

Q30. What if two nodes both think they are the Super-Peer at the same time (split-brain)? What If

If two phones think they are the Super-Peer, they will resolve it automatically. When a Super-Peer sees an election message, it sends a coordinator message to tell the other phone to stand down. Also, each cluster has a unique ID, so new phones will only connect to one Super-Peer.

Q31. What if a malicious node joins the network and sends fake DHT entries? What If

We only accept database updates from known active devices. Also, when downloading file fragments, the app verifies the SHA-256 hash of the file. If a rogue node sends fake or

modified data, the hash check fails and the app rejects the file.

Q32. What if a user's phone is stolen? Can the attacker read their files? What If

No. First, the phone only stores encrypted fragments, not whole files. Second, the private key is stored inside EncryptedSharedPreferences (encrypted using AES-256-GCM with a key from the Android Keystore TEE). The key is never written in plaintext on disk and is only decrypted in memory when needed, so it cannot be extracted by copying files.

Q33. What if the network has very high churn — many devices joining and leaving constantly? What If

If too many devices connect and disconnect, the network gets overloaded. We use a Circuit Breaker: if more than 30% of devices disconnect in 2 minutes, we stop trying to repair files immediately. We queue the repairs and wait until the network is stable again. This prevents network congestion.

Q34. What if two users upload the same file at the same time? What If

Both uploads will create entries based on the file's hash. The database uses timestamps (Last-Write-Wins) to decide which entry is the newest during sync. The system stores duplicate fragments in this prototype, but it does not cause any errors.

Q35. What if the Bloom filter used in gossip keeps growing and becomes inaccurate? What If

The Bloom filter is recreated from scratch during every gossip cycle, so it does not collect old data. If the number of files grows very large, the filter might become slightly less accurate. This only means we might send a few duplicate updates, but it does not cause any errors.

Q36. What if a node has very little battery left — does it still participate fully? What If

In our prototype, a low-battery node still behaves normally. However, in our simulations, we tested how battery drain works. In a production app, a phone with low battery would stop hosting new fragments and reduce its gossip activity to save power.

Q37. What if someone replays an old election message to confuse the network? What If

Every election message is signed and contains a timestamp. When a phone receives a message, it checks if it was sent in the last 30 seconds. If an attacker tries to resend an old message from 5 minutes ago, it is automatically rejected.

Q38. What if a peer wants to leave the network gracefully? What If

If the Super-Peer wants to leave, it sends an abdication message and waits 5 minutes before disconnecting. This gives the network time to elect a new leader. A regular node sends a leave message, and the Super-Peer immediately re-copies its fragments to other active phones.

Q39. What if the user's internet connection switches from Wi-Fi to 4G mid-transfer? What If

The app will reconnect to the relay. If a fragment transfer fails during the switch, the upload system retries it or sends it to a different node. We also have a 30-second delay before removing disconnected peers, which prevents false alarms.

Q40. What if you needed to scale to thousands of devices — would the current design hold? What If

No, the current relay server would become overloaded. To scale to thousands of devices, we would need to run multiple relay servers behind a load balancer. We would also need to organize Super-Peers into multiple tiers (like a hierarchy) to handle a global network.

Q41. What if a user loses their phone — can they recover their files on a new device? What If

Yes. When the user sets up their identity, they get a recovery code. The app encrypts their private keys with a key derived from a recovery password (using PBKDF2 with 100,000 iterations) and uploads it to Supabase. On a new device, entering the code and password decrypts the keys and restores access to the files.

Q42. P2P protocols are notoriously power-hungry. How does MobiCloud prevent draining the user's mobile battery? What If

We save battery in three ways: (1) we limit Gossip to talk to only 2 neighbors at a time; (2) the core math code is written in fast C++ (JNI) to save CPU power; and (3) we use a Circuit Breaker that stops network repair tasks if the network is too unstable, preventing useless battery drain.

Q43. Why did you choose k=3 data + n=2 parity for Reed-Solomon instead of other configurations like k=4 data + n=4 parity? Why

It is the best balance for mobile phones. With k=3 and n=2, we only use 1.67 times the file size in storage, and we can lose 2 nodes without losing the file. Using larger configurations like k=4, n=4 would require the phone to upload 8 files at the same time, which is too slow and heavy for mobile networks.

MobiCloud — Defense Preparation · 43 Questions (27 General/Why + 16 What If) · Anis Naoui · 2026